

ID:2403A51062
Batch:03
Name:S.Nithya Shree
Lab :03
Subject :AI &DL

```
import pandas as pd
import torch

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Load the dataset
df = pd.read_csv("/content/heart.csv")

# Display first 5 rows
print(df.head())

# 2. Separate input features and target
X = df.drop("target", axis=1)
y = df["target"]

# 3. Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# 4. Standardize the features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Convert data into PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(
    y_train.values,
    dtype=torch.float32
).reshape(-1, 1)

y_test = torch.tensor(
    y_test.values,
    dtype=torch.float32
).reshape(-1, 1)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
0	52	1	0	125	212	0	1	168	0	1.0	2
1	53	1	0	140	203	1	0	155	1	3.1	0
2	70	1	0	145	174	0	1	125	1	2.6	0
3	61	1	0	148	203	0	1	161	0	0.0	2
4	62	0	0	138	294	1	1	106	0	1.9	1

	ca	thal	target
0	2	3	0
1	0	3	0
2	0	3	0
3	1	3	0
4	3	2	0

Training data: torch.Size([820, 13])

Testing data: torch.Size([205, 13])

```
import torch
import torch.nn as nn

# Create ANN class
class ANN(nn.Module):

    def __init__(self, input_size):
        super(ANN, self).__init__()

        self.model = nn.Sequential(

            # Input layer → First hidden layer
            nn.Linear(input_size, 16),
            nn.ReLU(),

            # First hidden layer → Second hidden layer
            nn.Linear(16, 8),
            nn.ReLU(),

            # Second hidden layer → Output layer
            nn.Linear(8, 1),
            nn.Sigmoid()
        )

    # Forward propagation
    def forward(self, x):
        return self.model(x)

# Find number of input features
input_size = X_train.shape[1]

# Create the ANN model
model = ANN(input_size)

# Display the model
print(model)
```

```
ANN(  
    (model): Sequential(  
        (0): Linear(in_features=13, out_features=16, bias=True)  
        (1): ReLU()  
        (2): Linear(in_features=16, out_features=8, bias=True)  
        (3): ReLU()  
        (4): Linear(in_features=8, out_features=1, bias=True)  
        (5): Sigmoid()  
    )  
)
```

```
import torch  
import torch.nn as nn  
import torch.optim as optim  
  
# Loss function  
criterion = nn.BCELoss()  
  
# Optimizer  
optimizer = optim.Adam(  
    model.parameters(),  
    lr=0.001  
)  
  
# Number of training iterations  
epochs = 100  
  
# Store loss values  
losses = []  
  
# Training loop  
for epoch in range(epochs):  
  
    # 1. Forward pass  
    outputs = model(X_train)  
  
    # 2. Calculate loss  
    loss = criterion(outputs, y_train)  
  
    # 3. Clear previous gradients  
    optimizer.zero_grad()  
  
    # 4. Backpropagation  
    loss.backward()  
  
    # 5. Update weights  
    optimizer.step()  
  
    # Store loss  
    losses.append(loss.item())  
  
    # Display loss every 10 epochs  
    if (epoch + 1) % 10 == 0:  
        print(  
            "Epoch [{}/{}], Loss: {:.4f}".format(  

```

```
        epoch + 1,  
        epochs,  
        loss.item()  
    )  
)
```

```
Epoch [10/100], Loss: 0.6812  
Epoch [20/100], Loss: 0.6661  
Epoch [30/100], Loss: 0.6486  
Epoch [40/100], Loss: 0.6272  
Epoch [50/100], Loss: 0.6006  
Epoch [60/100], Loss: 0.5687  
Epoch [70/100], Loss: 0.5330  
Epoch [80/100], Loss: 0.4963  
Epoch [90/100], Loss: 0.4609  
Epoch [100/100], Loss: 0.4287
```

```
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    confusion_matrix,  
    classification_report  
)  
  
# Make predictions using test data  
with torch.no_grad():  
  
    # Get predicted probabilities  
    probabilities = model(X_test)  
  
    # Convert probabilities into 0 or 1  
    predictions = (probabilities >= 0.5).float()  
  
# Convert PyTorch tensors to NumPy arrays  
y_true = y_test.numpy()  
y_pred = predictions.numpy()  
  
# Calculate Accuracy  
accuracy = accuracy_score(  
    y_true,  
    y_pred  
)  
  
# Calculate Precision  
precision = precision_score(  
    y_true,  
    y_pred  
)  
  
# Calculate Recall  
recall = recall_score(  

```

```

        y_true,
        y_pred
    )

# Calculate F1 Score
f1 = f1_score(
    y_true,
    y_pred
)

# Display results
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall    :", recall)
print("F1 Score :", f1)

# Classification report
print("\nClassification Report:")
print(
    classification_report(
        y_true,
        y_pred
    )
)

# Confusion Matrix
cm = confusion_matrix(
    y_true,
    y_pred
)

print("\nConfusion Matrix:")
print(cm)

```

```

Accuracy : 0.7951219512195122
Precision: 0.7563025210084033
Recall    : 0.8737864077669902
F1 Score  : 0.8108108108108109

```

Classification Report:

	precision	recall	f1-score	support
0.0	0.85	0.72	0.78	102
1.0	0.76	0.87	0.81	103
accuracy			0.80	205
macro avg	0.80	0.79	0.79	205
weighted avg	0.80	0.80	0.79	205

Confusion Matrix:

```

[[73 29]
 [13 90]]

```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.plot(losses)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title("Training Loss vs Epoch")

plt.show()
```



